06 POINTERS AND REFERENCES • DETAILED SOLUTIONS

programming solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Programming Questions (Q27–Q32)

Solution to Q27 (In-Place Array Reversal Using Two Pointers)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* left = &arr[0];
    int* right = &arr[4];

    while (left < right) {
        int temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }

    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
// Expected Output: 50 40 30 20 10
```

Solution to Q28 (Array Min, Max, and Sum via `const int*` and References)

CPP

```cpp
#include <iostream>
using namespace std;

void compute_stats(const int* arr, int n, int& min_val, int& max_val, int& sum) {
    min_val = *arr;
    max_val = *arr;
    sum = 0;

    for (int i = 0; i < n; i++) {
        int current = *(arr + i);
        if (current < min_val) min_val = current;
        if (current > max_val) max_val = current;
        sum += current;
    }
}

int main() {
    int data[5] = {12, 5, 28, 9, 16};
    int mn = 0, mx = 0, total = 0;

    compute_stats(data, 5, mn, mx, total);
    cout << "Min: " << mn << ", Max: " << mx << ", Sum: " << total << endl;
    return 0;
}
// Expected Output: Min: 5, Max: 28, Sum: 70
```

Solution to Q29 (Array of Classroom `student` Objects Accessed via Pointer)

CPP

```cpp
#include <iostream>
using namespace std;

class student {
private:
    int roll;
    int marks;

public:
    void input(int r, int m) {
        roll = r;
        marks = m;
    }

    void display() {
        cout << "Roll: " << roll << ", Marks: " << marks << endl;
    }
};

int main() {
    student list[3];
    student* ptr = list;

    for (int i = 0; i < 3; i++) {
        (ptr + i)->input(i + 1, (i + 1) * 30);
    }

    for (int i = 0; i < 3; i++) {
        ptr->display();
        ptr++;
    }

    return 0;
}
/*
Expected Output:
Roll: 1, Marks: 30
Roll: 2, Marks: 60
Roll: 3, Marks: 90
*/
```

Solution to Q30 (Function Returning an Lvalue Reference `int&`)

CPP

```cpp
#include <iostream>
using namespace std;

int& get_element(int* arr, int index) {
    return *(arr + index);
}

int main() {
    int arr[3] = {10, 20, 30};
    get_element(arr, 1) = 99; // Modifies arr[1] directly!

    cout << arr[0] << " " << arr[1] << " " << arr[2] << endl;
    return 0;
}
// Expected Output: 10 99 30
```

Solution to Q31 (String Length and Vowel Count Using `const char*` and `*p++`)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    const char* text = "Mathematics";
    const char* p = text;
    int length = 0;
    int vowels = 0;

    while (*p != '\0') {
        char ch = *p++;
        length++;
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
            vowels++;
        }
    }

    cout << "Length: " << length << ", Vowels: " << vowels << endl;
    return 0;
}
// Expected Output: Length: 11, Vowels: 4
```

Solution to Q32 (Classroom `person` via Raw `new` / `delete` vs `std::unique_ptr`)

CPP

```cpp
#include <iostream>
#include <memory>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    void display() {
        cout << "Age: " << age << endl;
    }
};

int main() {
    // 1. Classroom raw pointer approach
    person* raw_p = new person;
    raw_p->input(20);
    raw_p->display();
    delete raw_p;
    raw_p = nullptr;

    // 2. Modern C++23 smart pointer approach
    unique_ptr<person> smart_p = make_unique<person>();
    smart_p->input(25);
    smart_p->display();

    return 0;
}
/*
Expected Output:
Age: 20
Age: 25
*/
```